BOOMI REFERENCE DESIGN · KAFKA EVENT STREAMING

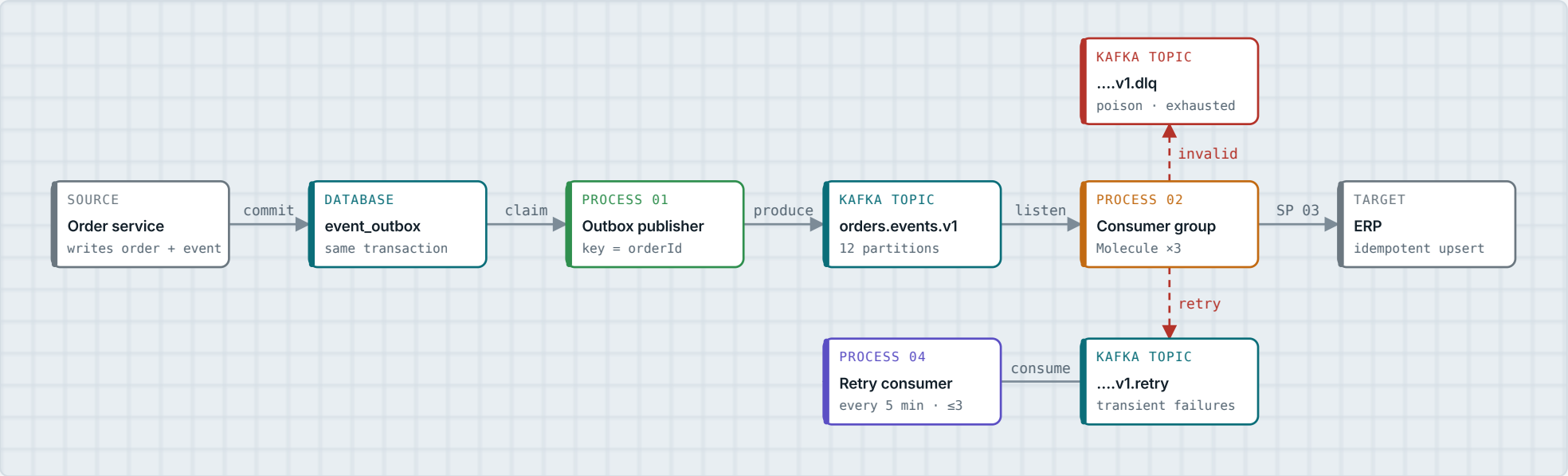
Order Event Stream

A Boomi event pipeline in four processes. A transactional outbox publishes order events to Kafka without losing or inventing any. A consumer group spread over Molecule nodes reads them in per-order sequence and applies each event exactly once to the target. Failures move to a retry topic and then a dead-letter topic, so one bad event never blocks its partition.

Start Logic / control Data shaping Connector Process call Error handling Stop / return

00 · ARCHITECTURE

Outbox → topic → consumer group

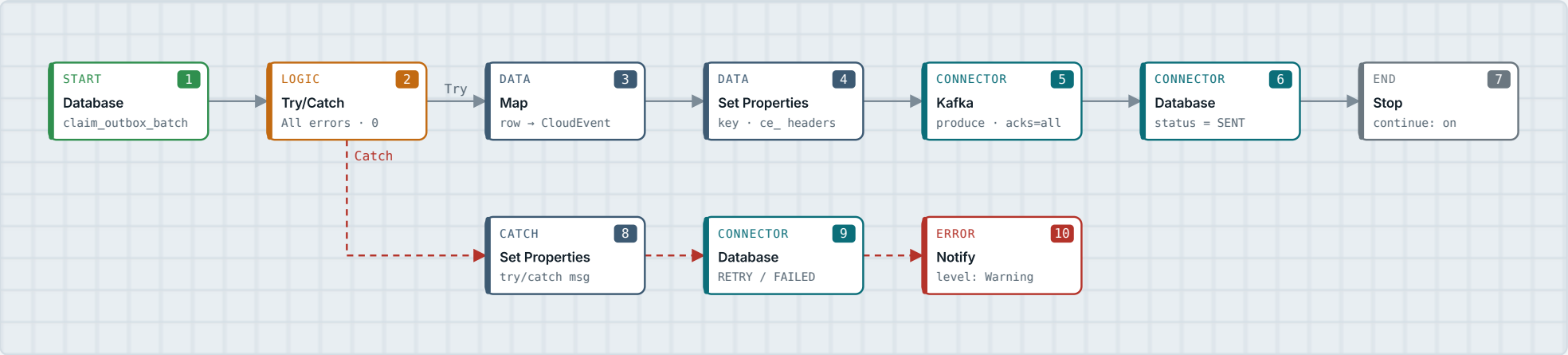


GUARANTEE	HOW	WHERE
No lost events	Business write and event row commit in one DB transaction; the publisher relays rows afterwards	Outbox + process 01
Per-order ordering	Kafka message key = orderId, so all events for one order land in one partition	Process 01, shape 4
Scale-out	One consumer group; Kafka spreads partitions over listeners on every Molecule node	Process 02
Exactly-once effect	At-least-once delivery + processed_events claim + idempotent upsert at the target	Sub process 03
No partition blocking	Failed events leave the main topic for retry / DLQ topics, and the offset moves on	Processes 02 and 04

01 · PROCESS

PR_EVT_Outbox_Publisher

Schedule: every 1 min | Allow Simultaneous Executions: Off | Purge Data Immediately: On



1 Start — Database get

START

PROPERTY	VALUE	WHY
SQL	`SELECT * FROM claim_outbox_batch(500) ORDER BY outbox_id`	Claims rows atomically, in creation order
Simultaneous executions	`Off`	A single publisher keeps per-order order across batches

> The schedule floor is one minute. For lower latency, have the writing process call this publisher right after its commit and keep the schedule as a sweeper.

2 Try/Catch

CONTROL

PROPERTY	VALUE	WHY
Failure Trigger / Retry	`All Errors / 0`	The next schedule run is the retry; attempts are counted in the row

3 Map — row → CloudEvent

DATA

PROPERTY	VALUE	WHY
Source / Destination	`DB outbox row / JSON ORD_Event_v1`	Envelope in section 05
id	`"evt-" + outbox_id`	Stable, so a re-send carries the same id
data	`payload column (already JSON)`	

4 Set Properties — key + headers

DATA

PROPERTY	VALUE	WHY
Kafka message key	`Profile Element › subject (orderId)`	Same key → same partition → ordered
Header ce_id / ce_type	`id / type from the envelope`	Consumers route and dedupe without parsing the body
Header traceparent	`Execution Property › Execution Id`	End-to-end tracing

> Kafka key and custom headers are set as Kafka connector document properties. Names differ slightly between connector versions.

5 Kafka — produce

CONNECTOR

PROPERTY	VALUE	WHY
Action / Topic	`Produce / orders.events.v1`	
acks	`all`	Written to every in-sync replica before success
enable.idempotence	`true`	Broker drops producer retries' duplicates
compression.type	`lz4`	
Connection	`SASL_SSL · credentials in environment extensions`	

6 Database — mark SENT

CONNECTOR

PROPERTY	VALUE	WHY
SQL	`UPDATE event_outbox SET status='SENT', sent_at=now() WHERE outbox_id=?`	

> A crash between 5 and 6 re-sends the event later. That's the reason the consumer deduplicates.

9 Database — mark RETRY / FAILED

CONNECTOR

PROPERTY	VALUE	WHY
SQL	`UPDATE … SET status = CASE WHEN attempts >= 10 THEN 'FAILED' ELSE 'RETRY' END, last_error=?`	Broker outage just backs up; nothing is lost

10 Notify

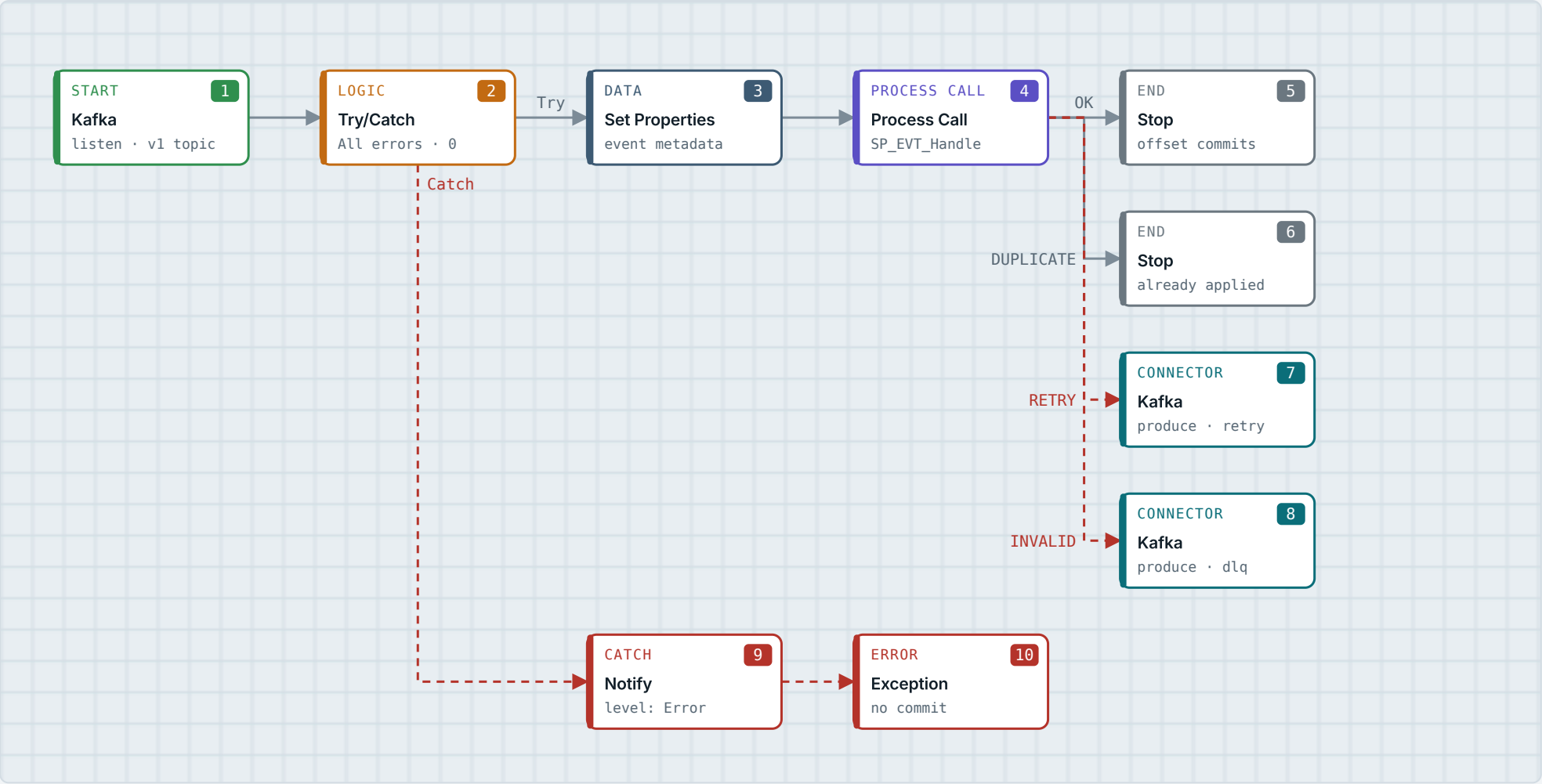
ERROR

PROPERTY	VALUE	WHY
Level / Message	`Warning / Outbox publish failed: {1}`	Alert separately on outbox rows older than 5 min

02 · PROCESS

PR_EVT_Order_Consumer

Process Mode: General | Molecule ×3 | Allow Simultaneous Executions: On



1 Start — Kafka listen

START

PROPERTY	VALUE	WHY
Topic	`orders.events.v1`	
Consumer group	`boomi-erp-sync`	Same group on every node, so partitions are shared out
Auto offset reset	`earliest`	A new group starts from retained history, not from now
Consumers per node	`partitions ÷ nodes (12 ÷ 3 = 4)`	More consumers than partitions sit idle
Commit	`After the execution succeeds`	At-least-once; check this setting on your connector version

2 Try/Catch

CONTROL

PROPERTY	VALUE	WHY
Failure Trigger / Retry	`All Errors / 0`	In-place retries stall the whole partition; use the retry topic instead

3 Set Properties — event metadata

DATA

PROPERTY	VALUE	WHY
DDP_EVENT_ID · DDP_EVENT_TYPE	`Kafka headers ce_id · ce_type`	
DDP_SRC_TOPIC · PARTITION · OFFSET	`Kafka connector document properties`	Go into DLQ headers for replay
DDP_RETRY_ATTEMPT	`Static: 0`	First pass

4 Process Call — SP_EVT_Handle

EXEC

PROPERTY	VALUE	WHY
Wait / Abort if fails	`On / Off`	Shared with the retry consumer
Return paths	`OK · DUPLICATE · RETRY · INVALID`	

7 Kafka — produce to retry topic

CONNECTOR

PROPERTY	VALUE	WHY
Topic	`orders.events.v1.retry`	
Key / body	`Original key / original body`	Unchanged, so the retry is identical
Headers added	`retry_attempt = 1, error_class, error_message, first_failure_at`	Same fields as the repo's DLQ envelope

8 Kafka — produce to DLQ

CONNECTOR

PROPERTY	VALUE	WHY
Topic	`orders.events.v1.dlq`	Poison pills skip retry entirely
Headers added	`error_class=POISON_PILL, original_topic, partition, offset, consumer_group`	Enough to replay and to find the source

9·10 Notify + Exception

ERROR

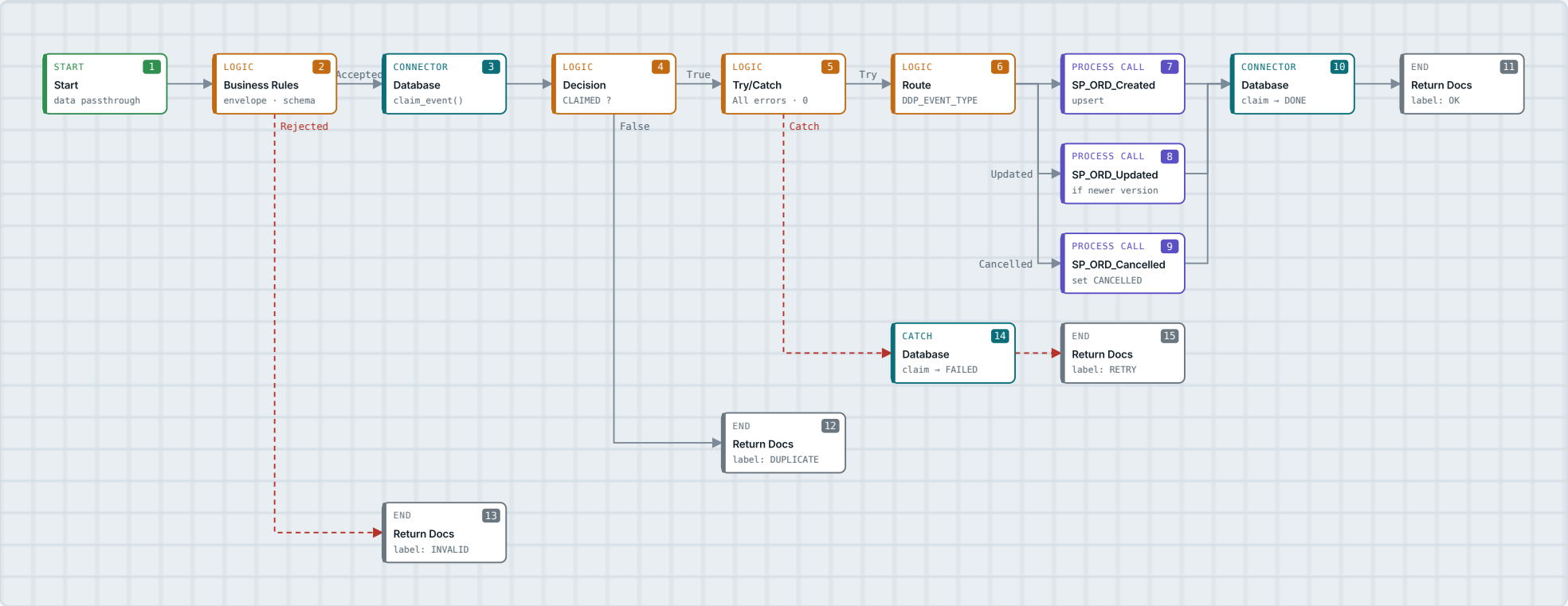
PROPERTY	VALUE	WHY
Reached when	`Producing to retry/DLQ itself failed`	Kafka is unreachable, so the event can't be parked
Exception	`Stop a single document: Off`	The execution fails, the offset isn't committed, and the event is redelivered. Nothing is lost.

03 · SUB PROCESS

SP_EVT_Handle

Start: Data Passthrough

Returns: OK · DUPLICATE · RETRY · INVALID



2 Business Rules — envelope + schema

CONTROL

RULE	CHECK	WHY
1	`id, type, subject, data present`	Malformed events are poison pills
2	`specversion = 1.0 and dataschema ends with /v1`	A v2 event on a v1 consumer gets parked, not mangled
3	`type in OrderCreated, OrderUpdated, OrderCancelled`	Unknown types go to INVALID

3 Database — claim event

CONNECTOR

PROPERTY	VALUE	WHY
SQL	`SELECT claim_event(?, 'erp-sync') AS claim`	CLAIMED or DUPLICATE, atomically

> Like any DB lookup, this replaces the document. Put a Branch or Document Cache around it so the event body continues on the CLAIMED path.

4 Decision — claimed?

CONTROL

PROPERTY	VALUE	WHY
First / Comparison / Second	`claim · Equal To · "CLAIMED"`	False → DUPLICATE, which is skipped silently

5 Try/Catch

CONTROL

PROPERTY	VALUE	WHY
Failure Trigger / Retry	`All Errors / 0`	The catch path releases the claim and asks for RETRY

6 Route — event type

CONTROL

PROPERTY	VALUE	WHY
Route by	`Document Property › DDP_EVENT_TYPE`	Header value, no body parse
Routes	`OrderCreated · OrderUpdated · OrderCancelled`	One handler sub process each

7–9 Process Call — type handlers

EXEC

HANDLER	TARGET OPERATION	IDEMPOTENCY
SP_ORD_Created	`Upsert sales order by orderId`	Upsert, never insert
SP_ORD_Updated	`Update where source_version < event version`	Stale events are ignored
SP_ORD_Cancelled	`Set status CANCELLED`	Repeating it changes nothing

> If a handler gets a permanent 4xx from the target, it should throw an Exception with message prefix INVALID: instead of failing silently. The catch path can then test the prefix and return INVALID instead of RETRY.

10·14 Database — close the claim

CONNECTOR

SHAPE	SQL	WHY
10 (success)	`UPDATE processed_events SET status='DONE', done_at=now() WHERE …`	
14 (catch)	`UPDATE processed_events SET status='FAILED' WHERE …`	Lets the retry re-claim the event

11–13·15 Return Documents

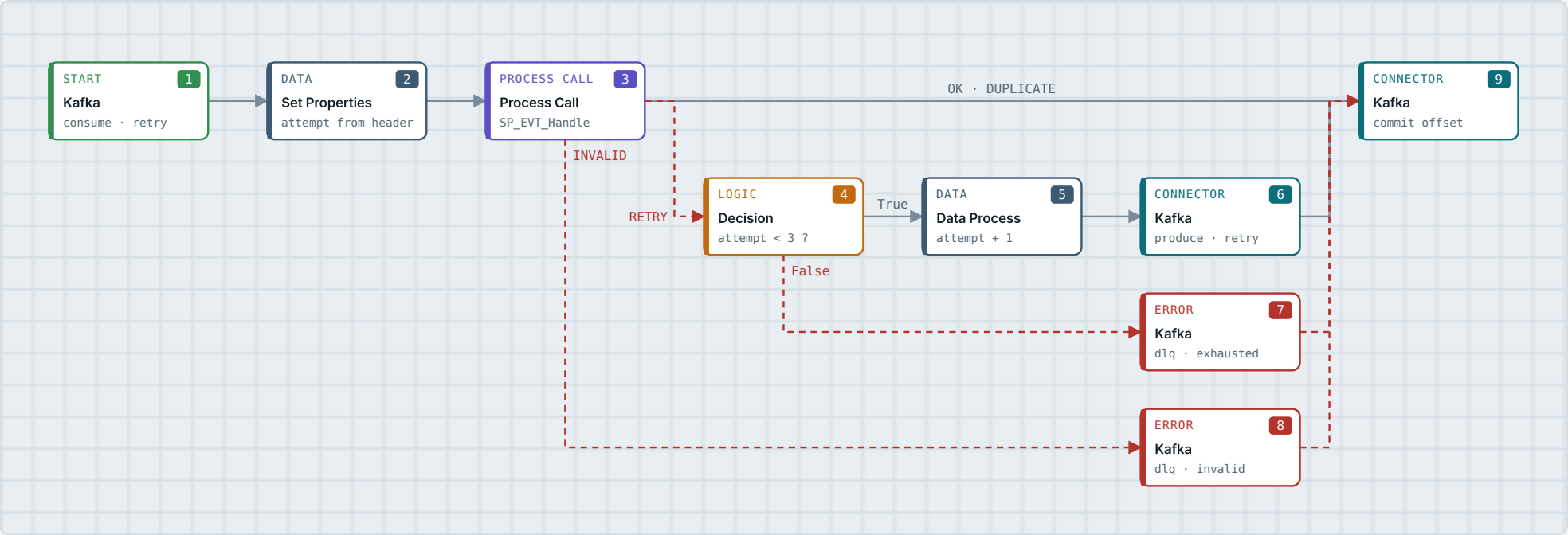
END

PROPERTY	VALUE	WHY
Labels	`OK · DUPLICATE · INVALID · RETRY`	Caller decides commit / retry topic / DLQ
RETRY path	`Set DDP_ERR_MSG from Try/Catch Message first`	Goes into the retry header

04 · PROCESS

PR_EVT_Retry_Consumer

Schedule: every 5 min | Allow Simultaneous Executions: Off | Max 3 attempts



1 Start — Kafka consume

START

PROPERTY	VALUE	WHY
Action / Topic	`Consume / orders.events.v1.retry`	Pull on a schedule, so the schedule interval is the back-off
Consumer group	`boomi-erp-sync-retry`	
Max messages	`500`	Bounded batch per run
Auto commit	`Off`	Shape 9 commits once each message has a home

4 Decision — attempts left?

CONTROL

PROPERTY	VALUE	WHY
First / Comparison / Second	`header retry_attempt · Less Than · 3`	False → DLQ with error_class=RETRY_EXHAUSTED

5 Data Process — attempt + 1

DATA

PROPERTY	VALUE	WHY
Processing Step	`Custom Scripting`	Increments the retry_attempt header DDP, like the DLQ replay counter in the pipeline design

9 Kafka — commit offset

CONNECTOR

PROPERTY	VALUE	WHY
Action	`Commit Offset`	Uses each document's topic/partition/offset properties
Placement	`After every path has re-published or finished`	A crash before this point re-reads the batch, and dedupe absorbs it

> Ordering trade-off: an event that goes to the retry topic can be overtaken by a later event for the same order. Handler rules like "update only if newer version" make that harmless. For flows where strict order matters more than throughput, park the whole key instead (a blocked_keys table checked in SP_EVT_Handle).

05 · CONTRACT

Topics and event envelope

TOPIC	PARTITIONS · RF	RETENTION	KEY	NOTES
orders.events.v1	12 · 3	7 days	orderId	min.insync.replicas=2; partitions cap consumer parallelism
orders.events.v1.retry	3 · 3	3 days	orderId	Drained every 5 min
orders.events.v1.dlq	3 · 3	30 days	orderId	Alert on depth > 0 for P1; replay after the fix

```
{
  "specversion": "1.0",
  "id": "evt-184467",                      // stable outbox id → dedupe key
  "source": "erp/order-service",
  "type": "OrderUpdated",
  "subject": "ORD-20431",                  // also the Kafka key
  "time": "2026-10-14T20:41:07Z",
  "dataschema": "https://schemas.example.com/orders/v1",
  "datacontenttype": "application/json",
  "data": { "orderId": "ORD-20431", "version": 7, "status": "SHIPPED", "lines": [ … ] }
}
```

> Breaking changes go to a new topic (orders.events.v2) with both versions published during migration, never into v1. If you run Confluent Schema Registry, register the JSON schema there and turn on compatibility checks.

06 · SHARED STATE

Outbox and processed events (PostgreSQL)

```
-- Producer side: written in the SAME transaction as the business change
create table event_outbox (
  outbox_id     bigserial primary key,
  aggregate_id  varchar(80)  not null,          -- orderId → Kafka key
  event_type    varchar(80)  not null,
  payload       jsonb        not null,
  created_at    timestamptz  not null default now(),
  status        varchar(10)  not null default 'NEW',   -- NEW | SENDING | RETRY | SENT | FAILED
  attempts      int          not null default 0,
  claimed_at    timestamptz,
  sent_at       timestamptz,
  last_error    text
);
create index ix_outbox_status on event_outbox(status, outbox_id);

create or replace function claim_outbox_batch(p_limit int)
returns setof event_outbox as $$
  update event_outbox o
     set status = 'SENDING', attempts = attempts + 1, claimed_at = now()
   where o.outbox_id in (
         select outbox_id from event_outbox
          where status in ('NEW', 'RETRY')
             or (status = 'SENDING' and claimed_at < now() - interval '5 minutes')   -- publisher crashed mid-batch
          order by outbox_id
          limit p_limit
          for update skip locked)
  returning o.*;
$$ language sql;

-- Consumer side
create table processed_events (
  consumer    varchar(60)  not null,
  event_id    varchar(80)  not null,
  status      varchar(12)  not null,              -- IN_PROGRESS | DONE | FAILED
  claimed_at  timestamptz  not null default now(),
  done_at     timestamptz,
  primary key (consumer, event_id)
);
```

```
create or replace function claim_event(p_event text, p_consumer text) returns text as $$
begin
  insert into processed_events(consumer, event_id, status)
  values (p_consumer, p_event, 'IN_PROGRESS')
  on conflict do nothing;
  if found then return 'CLAIMED'; end if;

  update processed_events set status = 'IN_PROGRESS', claimed_at = now()
   where consumer = p_consumer and event_id = p_event
     and (status = 'FAILED' or (status = 'IN_PROGRESS' and claimed_at < now() - interval '10 minutes'));
  if found then return 'CLAIMED'; end if;

  return 'DUPLICATE';
end $$ language plpgsql;

-- Housekeeping: keep processed ids at least as long as topic retention (7 d) plus margin
delete from processed_events where status = 'DONE' and done_at < now() - interval '14 days';
delete from event_outbox     where status = 'SENT' and sent_at < now() - interval '7 days';
```

07 · OPERATIONS

What to watch, and the Event Streams variant

SIGNAL	THRESHOLD	MEANING
Consumer lag (boomi-erp-sync)	`Growing for 10 min`	Add partitions/consumers, or the target is slow
Oldest NEW/RETRY outbox row	`> 5 min`	Publisher stuck or Kafka unreachable
Outbox rows in FAILED	`> 0`	10 publish attempts failed; investigate
Retry topic rate	`Spike above baseline`	Target system degrading
DLQ depth	`> 0 (P1)`	Triage, fix, replay to orders.events.v1

- **Boomi Event Streams** (Boomi's managed broker) fits the same design. Swap the Kafka shapes for the Event Streams connector's produce, listen and consume operations, keep the key by orderId and the subscription per consumer group. Check its built-in retry and dead-letter settings before adding your own retry topic.
- Replay a DLQ by re-producing its messages to orders.events.v1 after the fix. Because the consumer deduplicates on event id, replaying a whole batch is safe even if some succeeded.
- Never call slow external APIs inside the outbox transaction, and never write to Kafka from inside a DB transaction in Boomi. The outbox exists so you don't have to.